BMC Helix Technical Bootcamp - ITSM/ITOM Container

Lab 1: Verify the Environment Before ITOM Deployment

Lab 1: Verify the Environment Before ITOM Deployment 2
Task 1: Validate Installation Tools 2
Task 2: Validate Supporting Infrastructure 5

Lab 1: Verify the Environment Before ITOM Deployment

Objective: This lab will guide you through validating a Kubernetes environment for ITOM deployment and understanding the key components of the solution.

The steps provided in this lab document are simplified and tailored to our specific environment. Refer to the full product documentation for complete planning details. The Planning documentation provides details about container deployment architecture, system requirements, and sizing baselines:

https://docs.bmc.com/xwiki/bin/view/IT-Operations-Management/On-Premises-Deployment/BMC-Helix-IT-Operations-Management-Deployment/itomdeploy251/Planning/

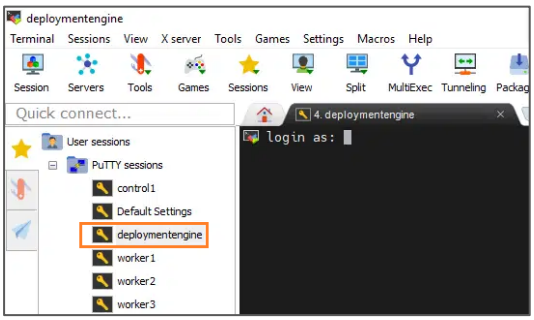
Task 1: Validate Installation Tools

In this task you will validate namespaces, metrics, SSH keys, and K9s.

1. Open **MobaXterm** to connect to the **deploymentengine** using these credentials:

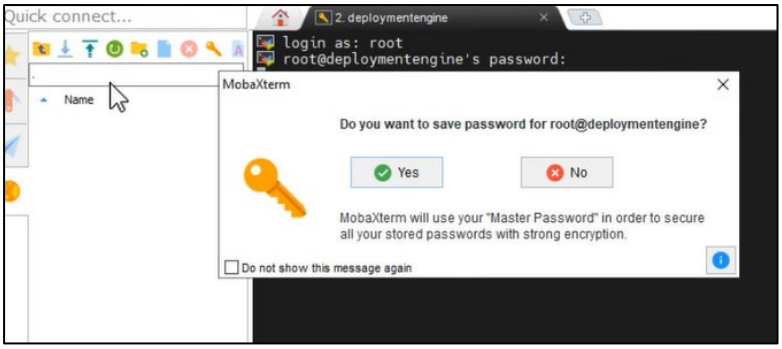
   Username = root

   Password = password



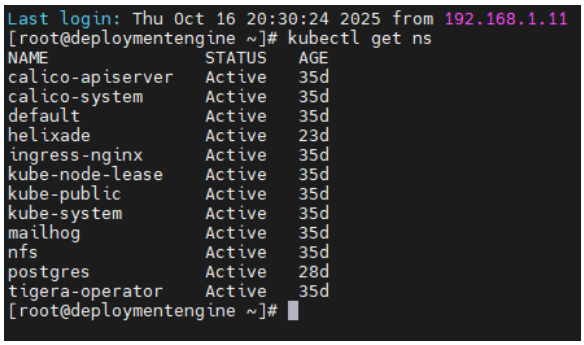
   Note: In this environment, we will use **deploymentengine** as the dedicated deployment host. This Linux host contains all the required files for the installation and upgrade used in this bootcamp. You can use PuTTY instead of MobaXterm if you prefer.

2. Click **No** on the save password option and select **Do not show this message again**.

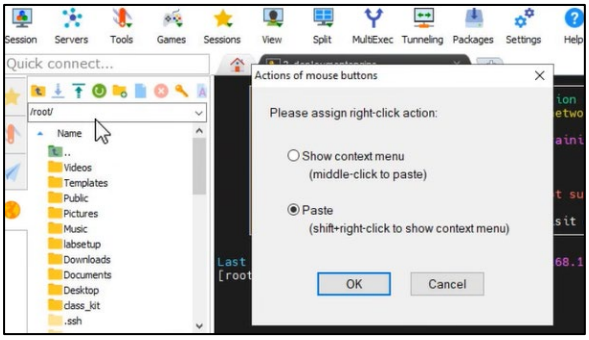


3. The **helixade** namespace has already been created. Verify the existing namespaces by running this command:

```
kubectl get ns
```

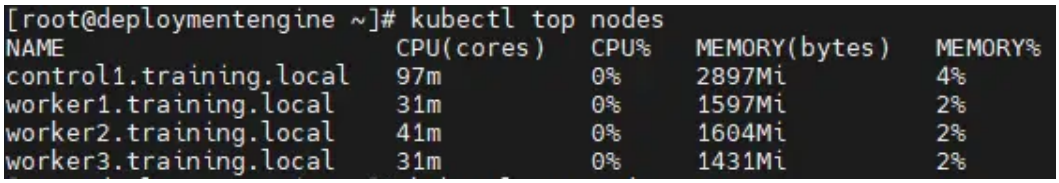


Note: If you copy and paste by right clicking into the terminal, you will be prompted to assign an action. If you want to have text be pasted in on right click, select **Paste** as the option.



4. **Kubernetes Metrics Service** is required for Horizontal Pod Autoscaling. To verify it's running, ensure you see output of pod metrics from this command:

```
kubectl top nodes
```



Note: Horizontal Pod Autoscaling is used by the BMC Helix Platform so that the product can scale based upon customer usage.

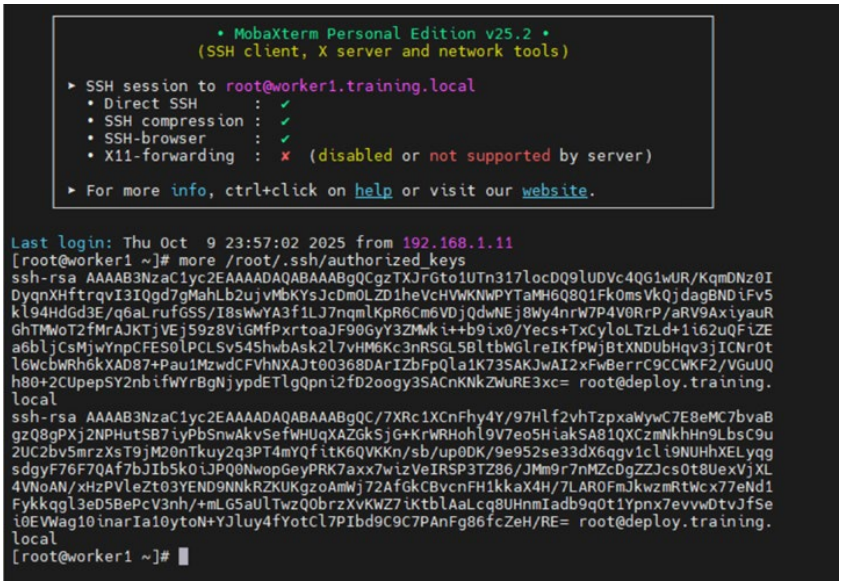
5. Use **MobaXterm** to connect to a **worker node (worker1, worker2, worker3)** using these credentials:

   Username = root

   Password = password

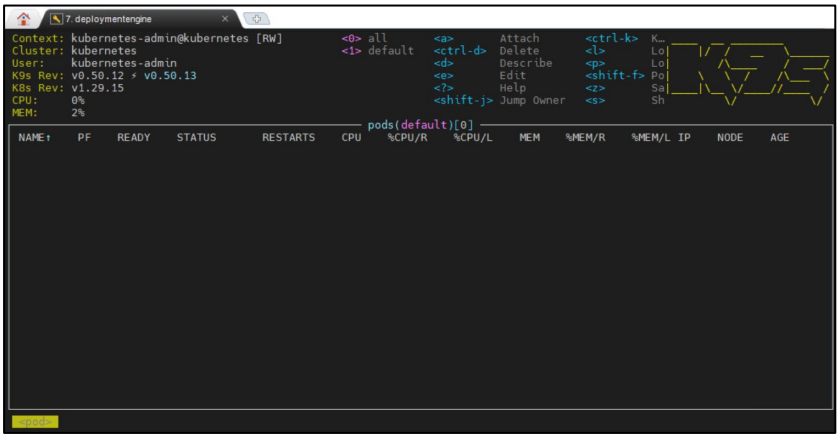
6. Run the following command on any of the worker nodes to verify SSH keys are already set up between the deployment workstation and the worker nodes:

```
more /root/.ssh/authorized_keys
```



7. K9s is already installed. It is a free terminal-based UI client to interact with your Kubernetes clusters. It is very useful to quickly view pod metrics, events and container logs. To launch K9s, from the **deploymentengine**, run the command:

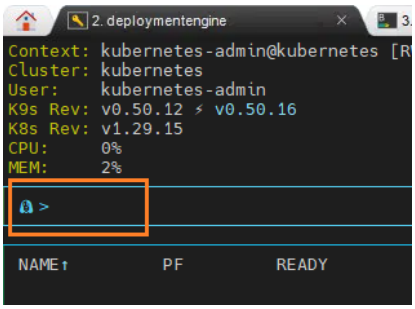
```
k9s
```



Note: The command line arguments are available here. A K9s YouTube tutorial is available here .

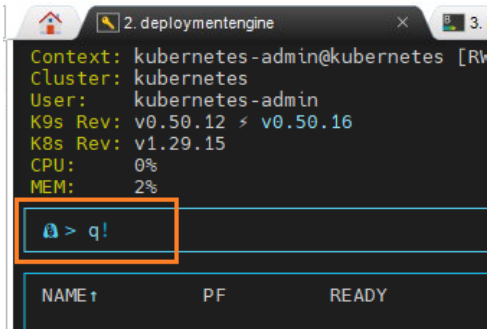
8. To exit **K9s**, type the following to open a small command prompt at the top of the K9s screen:

```
:
```



9. To return to your regular user session, type the following:

   q
   hit the ***Enter*** key



Task 2: Validate Supporting Infrastructure

In this task you will validate the environment by checking that the infrastructure is configured as required by verifying the following: container repository access, DNS, Discovery appliance, NFS, Ingress controller, HAProxy load balancer, and worker node memory settings.

1. From the **deployementengine**, verify you can log into the container repository by running this command:

   ```
   docker login deployementengine.training.local -u dev-itom-class -p Admin12345
   ```

   ```
   Last login: Mon Oct 13 18:25:52 2025 from 192.168.1.11
   [root@deployementengine ~]# docker login deployementengine.training.local -u dev-itom-class -p Admin12345
   WARNING! Using --password via the CLI is insecure. Use --password-stdin.
   Login Succeeded
   [root@deployementengine ~]#
   ```

   Note:
   - container repository = deployementengine.training.local
   - username = dev-itom-class
   - password = Admin12345

2. Verify that images are present in the repository by running this command to pull the ADE Infrastructure Clients container image. This is a common image used early in the deployment process.

   ```
   docker pull deployementengine.training.local/bmc/lp0lz:25100-v151-ade-infra-clients-alpine
   ```

```
[root@deploymentengine ~]# docker pull deploymentengine.training.local/bmc/lp0lz:25100-v151-ade-infra-clients-alpine
25100-v151-ade-infra-clients-alpine: Pulling from bmc/lp0lz
1f3e46996e29: Pull complete
94ee656eb977: Pull complete
0391260102b2: Pull complete
4f4fb700ef54: Pull complete
9ff7cbe4fd29: Pull complete
d596d98db2fc: Pull complete
38855aba0333: Pull complete
d9c37e5eb6a3: Pull complete
9674c33c226d: Pull complete
01cbd8baf7ce: Pull complete
703b72caf374: Pull complete
45fddd443efe: Pull complete
6a1e02bac3c7: Pull complete
7911a04924f4: Pull complete
6891a9388e5a: Pull complete
7faf6927f341: Pull complete
8a357b66a5f7: Pull complete
c7138957e06e: Pull complete
db47c636b75e: Pull complete
2299631dd7dd: Pull complete
137e33c01d55: Pull complete
8d605327c62a: Pull complete
14622bf13d91: Pull complete
Digest: sha256:ca2ccf924e38c397c8be9d24f52be01b29911f716ffb11dc79bfaf61d29c3588
Status: Downloaded newer image for deploymentengine.training.local/bmc/lp0lz:25100-v151-ade-infra-clients-alpine
deploymentengine.training.local/bmc/lp0lz:25100-v151-ade-infra-clients-alpine
[root@deploymentengine ~]#
```

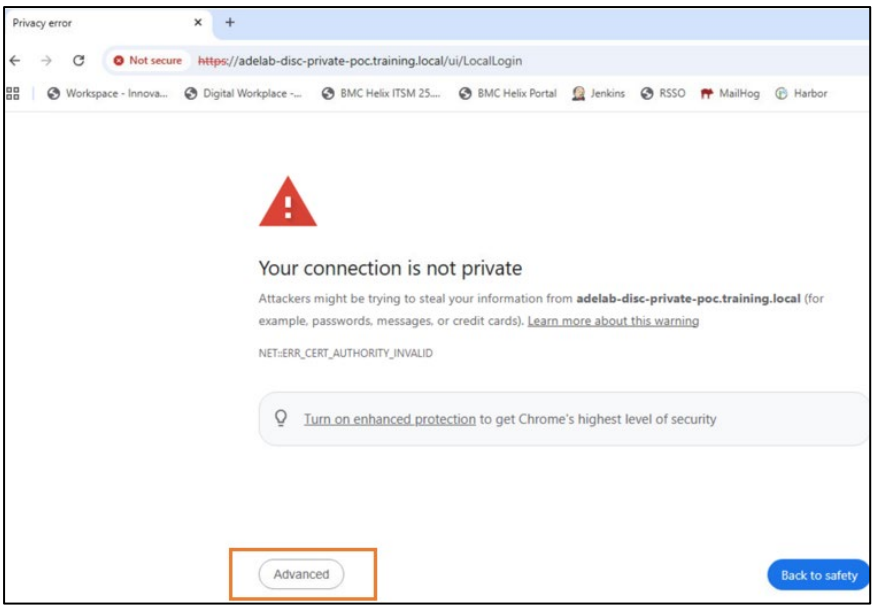
3. DNS service resolves hostnames to IP addresses as configured in the cluster. There is a DNS server on control1. The config files for the DNS server are in /var/named. The same IP address is used for all application hostnames except the Discovery appliance. Run this command on the **deploymentengine** to test name resolution and see where the HAProxy load balancer is installed:

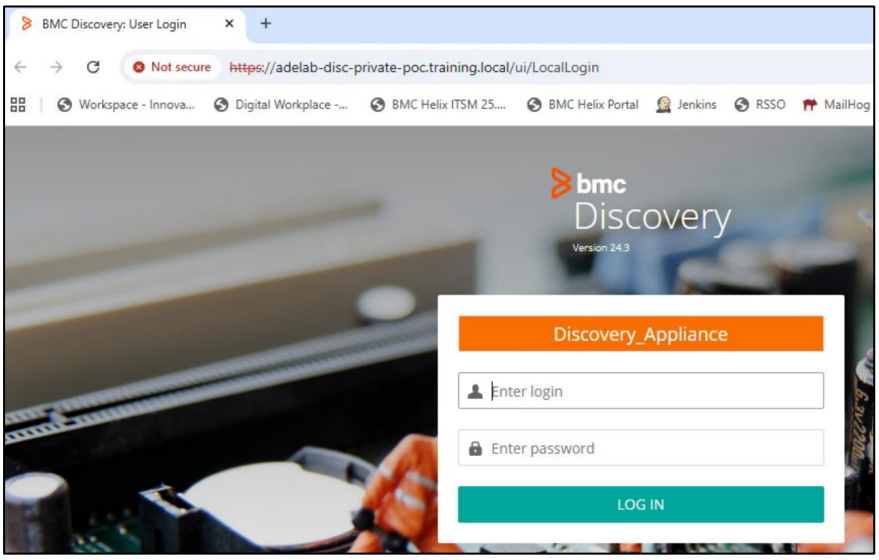
   `nslookup adelab-disc-private-poc.training.local`

```
[root@deploymentengine ~]# nslookup adelab-disc-private-poc.training.local
Server:         192.168.1.11
Address:        192.168.1.11#53

Name:   adelab-disc-private-poc.training.local
Address: 192.168.1.15

[root@deploymentengine ~]#
```

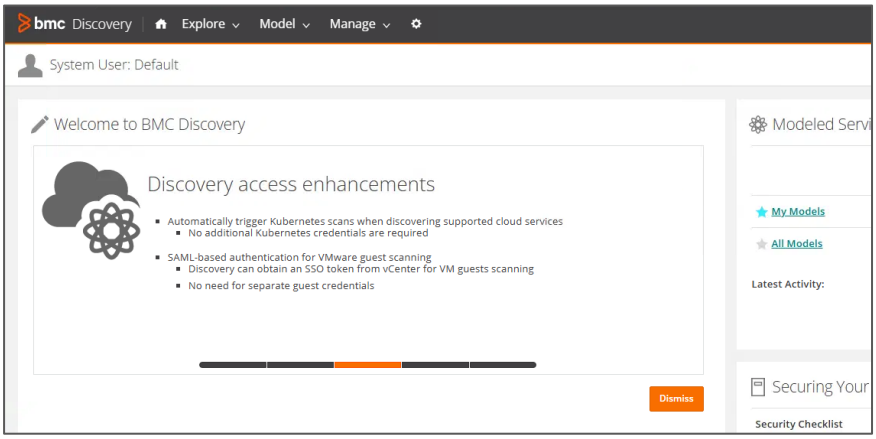
4. Verify the **Discovery appliance** is running by opening a Chrome browser on your landing desktop and using the URL listed below. You will get a "**Your connection is not private**" page that can be bypassed by clicking **Advanced** and then ***Proceed to adelab-disc-private-poc.training.local (unsafe).***

   `https://adelab-disc-private-poc.training.local/ui/LocalLogin`





5. Log into **Discovery** using these credentials: **system/bmcAdm1n!**



Note: A Discovery appliance is optional for Innovation Suite only deployments. The Discovery Appliance is used for several tasks, including integrating the Smart Graph function. The url for the Discovery Appliance is calculated from a number of configuration variables.

6. Using **MobaXterm** connect to **control1** using the credentials **root/password**.

7. Validate that the NFS service is running on **control1** using the following command:

```
systemctl status nfs-server
```

```
[root@control1 ~]# systemctl status nfs-server
● nfs-server.service - NFS server and services
     Loaded: loaded (/usr/lib/systemd/system/nfs-server.service; enabled; preset: disabled)
    Drop-In: /run/systemd/generator/nfs-server.service.d
             └─order-with-mounts.conf
     Active: active (exited) since Mon 2025-10-13 17:49:13 IST; 2h 39min ago
       Docs: man:rpc.nfsd(8)
             man:exportfs(8)
    Process: 1053 ExecStartPre=/usr/sbin/exportfs -r (code=exited, status=0/SUCCESS)
    Process: 1062 ExecStart=/usr/sbin/rpc.nfsd (code=exited, status=0/SUCCESS)
    Process: 1146 ExecStart=/bin/sh -c if systemctl -q is-active gssproxy; then systemctl re
   Main PID: 1146 (code=exited, status=0/SUCCESS)
        CPU: 19ms

Oct 13 17:49:12 control1.training.local systemd[1]: Starting NFS server and services...
Oct 13 17:49:13 control1.training.local systemd[1]: Finished NFS server and services.
```

8. The Ingress Controller is already configured in this system. Identify the configmap name by running this command from the **deploymentengine**:

```
kubectl -n ingress-nginx get cm
```

```
[root@deploymentengine ~]# kubectl -n ingress-nginx get cm
NAME                       DATA   AGE
ingress-nginx-controller   8      32d
kube-root-ca.crt           1      32d
[root@deploymentengine ~]#
```

9. You will need to know the **ingress class** for configuring the installer in the next lab. To obtain it, run the following command and look for the line **"- --ingress-class="**:

```
kubectl -n ingress-nginx get ds ingress-nginx-controller -o
jsonpath='{.spec.template.spec.containers[].args}' | jq
```

```
[root@deploymentengine ~]# kubectl -n ingress-nginx get ds ingress-nginx-controller -o jsonpath='{.spec.template.spec.containe
rs[].args}' | jq
[
  "/nginx-ingress-controller",
  "--publish-service=$(POD_NAMESPACE)/ingress-nginx-controller",
  "--election-id=ingress-nginx-leader",
  "--controller-class=k8s.io/ingress-nginx",
  "--ingress-class=nginx",
  "--configmap=$(POD_NAMESPACE)/ingress-nginx-controller",
  "--validating-webhook=:8443",
  "--validating-webhook-certificate=/usr/local/certificates/cert",
  "--validating-webhook-key=/usr/local/certificates/key",
  "--enable-metrics=false",
  "--default-ssl-certificate=ingress-nginx/my-tls-secret"
]
```

10. The HAProxy load balancer runs on **control1**, which means all the required On-Premise hostnames have aliases to the same IP address. The configurations reside in this file in the following location: **/etc/haproxy/haproxy.cfg**. Using MobaXterm connect to **control1** using the credentials **root/password**.

11. Change directory to location of the **haproxy.cfg** file:

```
cd /etc/haproxy
```

12. View the contents of the **haproxy.cfg** file by running this command:

```
more haproxy.cfg
```

```
global
    # to have these messages end up in /var/log/haproxy.log you will
    # need to:
    #
    # 1) configure syslog to accept network log events.  This is done
    #    by adding the '-r' option to the SYSLOGD_OPTIONS in
    #    /etc/sysconfig/syslog
    #
    # 2) configure local2 events to go to the /var/log/haproxy.log
    #   file. A line like the following can be added to
    #   /etc/sysconfig/syslog
    #
    #    local2.*                       /var/log/haproxy.log
    #
    log         127.0.0.1 local2
    chroot      /var/lib/haproxy
    pidfile     /var/run/haproxy.pid
    maxconn     8000
    nbproc     8
    #nbthread     8
    cpu-map    auto:8/1-8 0-7
    user        haproxy
    group       haproxy
    daemon
    # turn on stats unix socket
    stats socket /var/lib/haproxy/stats
    # utilize system-wide crypto-policies
    ssl-default-bind-ciphers PROFILE=SYSTEM
    ssl-default-server-ciphers PROFILE=SYSTEM
    tune.ssl.default-dh-param 2048
defaults
    option tcplog
    mode tcp
    log                     global
```

13. Press the space bar to review the contents of the file. It will automatically exit the file once you reach the end or you could type **q** to quit.

14. HaProxy listens on port 443/tcp (HTTPS). In a customer's environment you will need to know what ports your Ingress controllers are listening on to be able to configure the relevant load balancer, no matter which model of load balancer you are using. This ingress controller setup listens on port 32641 on each worker instance. To see the ports on the ingress-nginx-controller run this command on the **deploymentengine**:

```
kubectl get all -n ingress-nginx
```

```
[root@deploymentengine ~]# kubectl get all -n ingress-nginx
NAME                                        READY   STATUS      RESTARTS   AGE
pod/ingress-nginx-admission-create-jnpth    0/1     Completed   1          32d
pod/ingress-nginx-admission-patch-tfcjk     0/1     Completed   2          32d
pod/ingress-nginx-controller-bnnw2          1/1     Running     9          27d
pod/ingress-nginx-controller-dfwc2          1/1     Running     13         32d
pod/ingress-nginx-controller-qt6st          1/1     Running     13         32d
pod/ingress-nginx-controller-sdp74          1/1     Running     13         32d

NAME                                         TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
service/ingress-nginx-controller             LoadBalancer   10.104.223.225   <pending>     80:32640/TCP,443:32641/TCP   32d
service/ingress-nginx-controller-admission   ClusterIP      10.104.204.175   <none>        443/TCP                      32d

NAME                                       DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR            AGE
daemonset.apps/ingress-nginx-controller    4         4         4       4            4           kubernetes.io/os=linux   32d

NAME                                       COMPLETIONS   DURATION   AGE
job.batch/ingress-nginx-admission-create   1/1           3m46s      32d
job.batch/ingress-nginx-admission-patch    1/1           4m         32d
```

15. All worker nodes need a specific setting for vm max memory settings. This has already been configured on the worker nodes. You can verify that by running the following command on each worker node:

```
sysctl vm.max_map_count
```

```
[root@worker1 ~]# sysctl vm.max_map_count
vm.max_map_count = 262144
[root@worker1 ~]#
```

16. Close your sessions to control1 and the worker nodes but keep the connection to **deploymentengine** ready for the next lab.